

# VoiceDesk — architecture plan

Living document. Every sentence here is true of the current code, or it gets corrected. History lives in git, not in this file.

**Repository:** VoiceDesktopElectron · **Product name:** VoiceDesk · **Platform:** macOS (Apple Silicon and Intel) · **Canonical version:** the `version` field of `package.json`.

---

## 1. What this is

---

A desktop window with one control. You **hold** a button (or the `Space` key while the window has focus), speak, and release. Your words appear as text. The text is handed to a **coding agent running through its own CLI** — not the model API — which reads and writes Markdown files in a `notes/` folder beside the app. The agent's reply appears in the window, and can be spoken back.

Two sentences define the whole product:

- *"add milk to my shopping list"* → the agent creates or edits `notes/shopping.md`.
- *"what's on my list?"* → the agent answers from that file.

### Non-goals

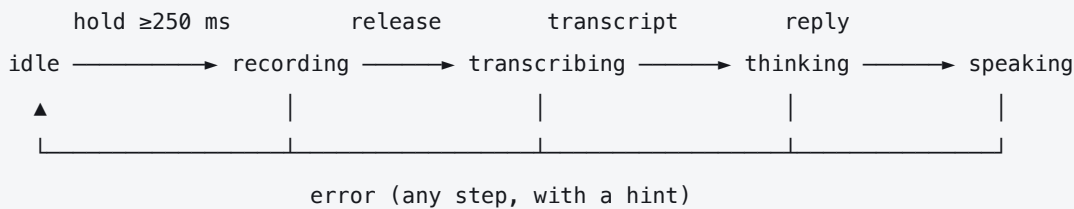
Named so they are not rediscovered as missing features:

- **Not a chat app.** One turn at a time. No conversation history UI beyond the current turn and a short scrollbar.
  - **Not a notes editor.** The app never writes `notes/` itself. Every byte in there is written by the agent, because "the agent can read and write files" is the thing being demonstrated.
  - **Not multi-agent, not multi-window, not cross-platform.** One window, one agent, macOS.
  - **Not a background dictation daemon.** Push-to-talk works while the window has focus. §6 records what that costs and what the alternative would have cost.
- 

## 2. The turn

---

The entire product is one flow, and it is a state machine. Every rule below hangs off it.



```
// src/domain/model/turn.ts
export type TurnState =
  | { k: 'idle' }
  | { k: 'recording'; startedAt: number; level: number }
  | { k: 'transcribing' }
  | { k: 'thinking' }
  | { k: 'speaking' }
  | { k: 'error'; failure: TurnFailure }
```

**One tagged union, not four booleans.** `isRecording && !isTranscribing` invites a state that must not exist; the union makes "recording while thinking" unrepresentable and gives the UI a single value to render.

Edge rules that belong to the machine, not to the UI:

- a hold shorter than **250 ms** is discarded — an accidental tap otherwise produces an empty transcript and a confused agent turn;
- key **repeat** is ignored, and key-down is ignored unless the state is `idle`;
- `pointerup`, `pointerleave`, `pointercancel` and window `blur` **all** end the hold, and the button uses `setPointerCapture` — a recording that never stops is the bug this prevents;
- every turn ends with `stream.getTracks().forEach(t => t.stop())`, or the macOS recording indicator stays lit and the user stops trusting the app.

### 3. Architecture: the process split *is* the layer boundary

Electron already runs three processes with different privileges. Drawing application layers anywhere else means maintaining two boundaries that will disagree.

| process         | privilege            | is the layer                                               | may import                                 |
|-----------------|----------------------|------------------------------------------------------------|--------------------------------------------|
| <b>main</b>     | full Node + OS       | infrastructure <b>and</b> composition root — "the backend" | anything                                   |
| <b>preload</b>  | bridge only          | the contract                                               | <code>electron</code> + shared types       |
| <b>renderer</b> | web page             | presentation                                               | shared types and the bridge — nothing else |
| <b>domain</b>   | <i>not a process</i> | policy: use cases, models, ports                           | <b>nothing</b> platform-shaped             |

`domain/` is plain TypeScript that would run in a browser, in a test, or in a CLI. It is why the tests need no Electron at all.

## Folder layout

```
src/
  domain/
    model/turn.ts           TurnState, TurnFailure
    model/transcript.ts     Transcript
    model/agent-reply.ts    AgentReply
    ports/Transcriber.ts    interface Transcriber
    ports/AgentRunner.ts    interface AgentRunner
    ports/SpeechSynthesizer.ts interface SpeechSynthesizer
    usecases/runVoiceTurn.ts audio → transcript → reply (takes ports as arguments)
  infrastructure/
    transcribe/WhisperCppTranscriber.ts
    transcribe/wav.ts        Float32Array → 16-bit PCM WAV (pure)
    agent/ClaudeCliAgentRunner.ts
    speech/MacSaySynthesizer.ts
    process/resolveBinary.ts PATH-independent binary lookup
  main/
    index.ts                 app lifecycle, the single window
    composition-root.ts      the ONLY place an adapter is constructed
    ipc.ts                   handlers: unwrap → use case → wrap
  preload/index.ts           contextBridge surface
  renderer/
    i18n/                   t(), en.json
  shared/ipc.ts              channel names + payload schemas – imported by all three
```

**The dependency rule, in one line:** arrows point inward. `infrastructure` imports `domain`; the composition root imports both; `domain` imports nothing from the platform.

No `utils/`, no `helpers/`, no root `types/`. A folder with no dependency direction collects everything and the arrows disappear.

## The rule is enforced by a test, not by this paragraph

```
// test/architecture.test.ts
const FORBIDDEN = /from\s+['"](electron|node:|fs|path|child_process|os)/
test('domain/ stays pure', () => {
  const files = globSync('src/domain/**/*.ts')
  expect(files.length).toBeGreaterThan(0) // the glob itself is a failure mode
  expect(files.filter(f => FORBIDDEN.test(read(f)))).toEqual([])
})
```

The gate is proven by breaking it: add a forbidden import, watch it redden, remove it. A gate nobody has seen fail is not evidence. The non-empty assertion is there because the classic silent pass is a glob that matched nothing.

Composition root: exactly one

```
// src/main/composition-root.ts
export function buildApp(env: Env) {
  const transcriber: Transcriber      = new WhisperCppTranscriber(env.whisperBin, env.modelPath)
  const agent: AgentRunner            = new ClaudeCliAgentRunner(env.agentBin, env.notesDir)
  const voice: SpeechSynthesizer      = new MacSaySynthesizer(env.voice)
  return { runVoiceTurn: makeRunVoiceTurn({ transcriber, agent, voice }) }
}
```

Selection happens **by outcome** — platform, env var, availability — in this one file. A new `WhisperCppTranscriber()` anywhere else is a dead seam. IPC handlers are adapters too: unwrap, call the use case, wrap. A handler containing business logic has moved policy into the framework.

4. The three seams

Each port is named for its **role**; each adapter for its **technology**. That naming is what tells you which side of the line a file is on.

| port (domain)     | adapter that ships                                            | second implementation, in the repo on day one                                            |
|-------------------|---------------------------------------------------------------|------------------------------------------------------------------------------------------|
| Transcriber       | WhisperCppTranscriber                                         | the test double — every domain test needs one, because the real adapter spawns a process |
| AgentRunner       | ClaudeCliAgentRunner , and <b>only</b> <code>claude -p</code> | the test double, same reason                                                             |
| SpeechSynthesizer | MacSaySynthesizer                                             | the null implementation used when speech is off, plus the test double                    |

**Only one agent CLI is implemented.** `codex exec` and `cursor-agent -p` are future work, not scaffolding to be written now.

That makes the usual objection fair, so it gets answered here rather than in review: *an interface with one implementation is an over-build*. It would be — if the count were one. It is not. Every adapter above reaches the OS, so **every test of the use case needs a substitute**, and that substitute is a real second implementation living in this repository from the first test onward. The port is what makes the domain testable without a microphone, a model file and a spawned agent; the fact that it also makes a second CLI cheap later is a side effect, not the argument.

The litmus stays sharp for everything else: if a future port cannot name its second implementation — a test double counts, "someday" does not — it does not get written.

```
// src/domain/ports/AgentRunner.ts
export interface AgentRunner {
  run(prompt: string, ctx: TurnContext): Promise<AgentReply>
}
```

The use case says "*ask the agent*". If the string `-p` appears anywhere outside `ClaudeCliAgentRunner`, the seam has leaked.

---

## 5. The agent CLI is a process boundary, and gets network-call treatment

---

The CLI already carries the agent loop, the file-editing tools, permission gating and session state. Calling it means shipping a harness rather than rebuilding a chat client. The cost is a dependency on a program that can hang, die, print garbage, or do more than it was asked.

### 5.1 `execFile` with an argv array — never a shell

```
execFile(bin, args, { cwd: notesDir, timeout: 90_000, maxBuffer: 8 * 1024 * 1024 }) // ✓
exec(`${bin} -p "${text}"`) // ✗
```

A dictated sentence must never reach a shell parser. "`add milk; rm -rf ~`" is one apostrophe away from ordinary speech. An argv array has no quoting to get wrong.

It also removes the login-shell surprise, and on this machine that surprise is **already present**: `claude` resolves to a *zsh function*, not to a binary. An interactive shell sees the function; `execFile` never will. §5.5 is the consequence.

### 5.2 The flags, and why each one is there

Every flag is a value chosen here, not a default inherited from someone else. Verified against the CLI actually installed (2.1.263) — the version is recorded in the README, because these are another program's flags and they can move.

| flag                                                                     | why                                                                                                                                                                |
|--------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>-p &lt;prompt&gt;</code>                                           | non-interactive: print the answer and exit                                                                                                                         |
| <code>--output-format json</code>                                        | a machine-readable envelope. Text output is for humans and changes without warning                                                                                 |
| <code>--allowedTools</code><br><code>Read,Write&gt;Edit,Glob,Grep</code> | exactly the capability the feature needs — read and write notes                                                                                                    |
| <code>--restricted</code>                                                | removes the built-in command-running tools and web fetch entirely. Belt to the allowlist's braces                                                                  |
| <code>--add-dir &lt;notesDir&gt;</code>                                  | states the writable area explicitly rather than relying on <code>cwd</code> alone                                                                                  |
| <code>--permission-mode acceptEdits</code>                               | file edits inside <code>notes/</code> proceed without a prompt. <b>Not</b> <code>bypassPermissions</code> , which would drop the boundary this app is built around |
| <code>--strict-mcp-config</code>                                         | no MCP servers are inherited from the operator's machine — behaviour that depends on whose config is present is not behaviour that can be tested                   |
| <code>--session-id &lt;uuid&gt; / --resume &lt;uuid&gt;</code>           | session continuity, carried explicitly (§5.4)                                                                                                                      |
| <code>--append-system-prompt &lt;text&gt;</code>                         | tells the agent the notes conventions: one Markdown file per subject, kebab-case names, answer in one short paragraph                                              |

Deliberately **not** granted: `Bash` and every other command-running tool. "The agent might need it" trades the entire permission boundary for a maybe.

### 5.3 Parse the reply, never cast it

```
const raw = JSON.parse(stdout)
const reply = AgentReplySchema.parse(raw)    // zod, at the seam
```

A field that stops arriving must fail loudly at the boundary, not surface as `undefined` three layers up.

### 5.4 Session continuity is carried, not hoped for

"Add milk to my list" and "what's on my list?" are two spawns. **The decision: the app generates a UUID per app session, passes `--session-id` on the first turn and `--resume` on every turn after.** A resumed session the CLI has forgotten is a *normal outcome*, not a crash: the adapter retries once as a fresh session and says so in the reply metadata. The alternative — a blank agent re-reading `notes/` every turn — also works, and is what the fallback lands on.

### 5.5 Three distinguishable outcomes, never two

| outcome                         | meaning                          | what the user is told                                             |
|---------------------------------|----------------------------------|-------------------------------------------------------------------|
| exit 0 + parseable stdout       | success                          | the reply                                                         |
| non-zero, or unparseable stdout | the agent failed                 | trimmed stderr, and that it was the agent                         |
| ENOENT                          | setup failure, not agent failure | "c laude was not found — install it, or set VOICEDesk_AGENT_BIN " |
| timeout                         | the call could not end           | the child is killed and the turn fails cleanly                    |

The third row is the one that matters most here. An Electron app launched from Finder inherits a **minimal** PATH that does not include `~/local/bin` or `/opt/homebrew/bin`, so a binary that works in the terminal is missing in the app. `resolveBinary` therefore checks, in order: an explicit env var, a settings value, then a list of known install locations — and when all fail it reports a *setup* problem with the exact fix. Collapsing that into "the agent failed" sends the user debugging the wrong thing.

## 6. Voice in: local Whisper

**Decision:** `whisper.cpp` (the `whisper-cli` binary), running locally, behind the Transcriber port.

Why, against the alternatives the brief allows:

| option                        | verdict                                                                                                                                                                                                |
|-------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>whisper.cpp local</b>      | ✅ no API key, no account, no network, no per-minute cost; already present on this machine via Homebrew; a reviewer's setup is two <code>brew install</code> lines and one model download               |
| hosted Whisper / Deepgram API | a reviewer must supply their own key before the app does anything. Good quality, worse "runs on a clean machine"                                                                                       |
| macOS on-device dictation     | zero setup and excellent, but reaching <code>SFSpeechRecognizer</code> from Electron needs a native helper binary — an hour of Swift and a build step, for a capability the first row already provides |

### The audio path

Capture lives in the **renderer**, because that is where the browser media stack is and it hands back the level meter for free.

```
getUserMedia → AudioContext({ sampleRate: 16000 }) → AudioWorklet → Float32Array (mono, 16 kHz)
  → ArrayBuffer over IPC → main → wav.ts (16-bit PCM) → temp file → whisper-cli → transcript
```

- **16 kHz is requested at the source**, not resampled afterwards, because that is exactly what Whisper wants. This is also why there is **no ffmpeg dependency**: `MediaRecorder` would give WebM/Opus that something would then have to transcode.
- `AnalyserNode` drives a visible level meter. A meter is not decoration — it is the only evidence the user has that the microphone is live, and its absence is the most common reason a voice UI feels broken.

- The stream is acquired on the **first hold**, never at startup. Asking for the microphone before the user has pressed anything reads as spyware and burns the permission prompt.
- Microphone permission has **three** outcomes, and the denied-permanently one names the exact place to fix it: *System Settings* → *Privacy & Security* → *Microphone*.

## Voice out (the brief's stretch goal)

**Decided: in scope.** `say`, the macOS built-in, behind `SpeechSynthesizer`. Zero dependencies, on-device, one `execFile`, and the reply is spoken back after it is shown. It stays this cheap or it does not ship — a neural voice would be a second adapter, not a rewrite.

## 7. The IPC contract

The preload bridge is a wire format shared by three separately-compiled build outputs, and a trust boundary: one XSS in the app's own UI is a compromised client.

```
// shared/ipc.ts – the single declaration, imported by main, preload and renderer
export const CH = {
  transcribe: 'turn:transcribe',
  ask:        'turn:ask',
  speak:     'turn:speak',
  appInfo:    'app:info',
  state:      'turn:state',
} as const

export const TranscribeReq = z.object({
  pcm:          z.instanceof(ArrayBuffer).refine(b => b.byteLength <= 16 * 1024 * 1024),
  sampleRate:   z.literal(16000),
})

export const AskReq = z.object({ text: z.string().min(1).max(4000) })
export const AskRes = z.object({ reply: z.string(), notes: z.array(z.string()) })
```

Rules that follow from it:

- **one named method per message**; `ipcRenderer` is never exposed, and no bridge function takes a channel name from its caller. The bridge surface must be enumerable by reading the file;
- `invoke / handle` for request→reply, `send / on` only for main pushing state; every subscription hands back an **unsubscribe** or a remounting component leaks a listener;
- **every** inbound payload is `parse`d in main before it reaches a use case — shape validated, sizes bounded, rejected rather than coerced;
- errors cross as **typed data**, not thrown `Errors`, so the UI can tell "no microphone" from "agent timed out" from "model file missing":



```
export type TurnFailure =
  | { kind: 'mic-denied';    permanent: boolean }
  | { kind: 'setup';        what: 'agent-cli' | 'whisper' | 'model'; hint: string }
  | { kind: 'agent-failed';  stderr: string }
  | { kind: 'timeout';      afterMs: number }
  | { kind: 'empty-speech' }
```

---

## 8. Security switches, written down on purpose

---

```
new BrowserWindow({
  webPreferences: {
    preload: join(__dirname, '../preload/index.js'),
    contextIsolation: true,    // default since Electron 12 – asserted anyway
    sandbox: true,            // default since Electron 20 – asserted anyway
    nodeIntegration: false,    // default – asserted anyway
  },
})
```

They are the current defaults. Writing them down regardless costs three lines and is the difference between an XSS bug and remote code execution on the user's machine — a default nobody chose is a default anyone can change without a review noticing.

Also set deliberately: `will-navigate` and `setWindowOpenHandler` **deny by default**; a CSP with no `unsafe-eval` (in a desktop app that is a shell); a single-instance lock, because two copies writing the same `notes/` is a bug that will not reproduce.

---

## 9. Version marker

---

One SemVer number, in one place: the `version` field of `package.json`. It is read at build time and rendered in the window during development as `v0.1.0 · dev`, so a test run shows at a glance whether the build on screen is the one just changed.

**Bump policy for this project: every code change bumps the patch level**, which is stricter than the usual once-per-merge rule and is deliberate — the version badge is being used as a live signal during manual testing, so it has to move whenever the code does.

---

## 10. Localisation

---

English only today, structured so a second language costs a file and no refactor:

```
src/renderer/i18n/en.json      { "talk.hold": "Hold to talk", ... }
src/renderer/i18n/index.ts    t(key) — reads the active locale, falls back to en
```

No i18n library: one dictionary, one lookup function, no plural rules and no interpolation until something needs them. Adding `pl.json` and a locale switch is the whole cost of language two. Every user-visible string goes through `t()` from the first commit — retrofitting that is the expensive half, and it is the half being avoided here.

## 11. Tech stack, and why

Every version is the current stable at the time of writing, and each one is a choice.

| what                                            | version                              | why this and not the alternative                                                                                                                                                                                              |
|-------------------------------------------------|--------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Electron</b>                                 | 44.x                                 | the requirement is a desktop app driving local binaries and the file system; Electron is the shortest path from a web UI to that, and the version line carries <code>windowStatePersistence</code> and the reworked clipboard |
| <b>TypeScript</b>                               | 7.x                                  | <code>strict</code> . Types at the seams are the review this project cannot afford to run by hand                                                                                                                             |
| <b>electron-vite</b>                            | 5.x                                  | gives main / preload / renderer as three build outputs with three tsconfigs — the layer split, for free, instead of a hand-rolled build                                                                                       |
| <b>Vite</b>                                     | 8.x                                  | the renderer dev server and bundler <code>electron-vite</code> builds on                                                                                                                                                      |
| <b>React</b>                                    | 19.x                                 | the UI is one screen with one state machine; React's cost here is small and the component vocabulary matches the design deliverable                                                                                           |
| <b>zod</b>                                      | 4.x                                  | schema validation at the two seams that need it: IPC payloads and agent stdout. Chosen over hand-written type guards because a guard that drifts from its type compiles fine                                                  |
| <b>Vitest</b>                                   | 5.x                                  | runs the domain and adapter tests with no Electron at all; same config shape as the renderer build                                                                                                                            |
| <b>Playwright</b><br>( <code>_electron</code> ) | current                              | one smoke test that launches the real app. §12 explains why exactly one                                                                                                                                                       |
| <b>whisper.cpp</b>                              | Homebrew<br><code>whisper-cpp</code> | §6                                                                                                                                                                                                                            |
| <code>say</code>                                | macOS built-in                       | §6                                                                                                                                                                                                                            |
| <b>Node</b>                                     | 26.x                                 | the runtime the toolchain is pinned to; <code>engine-strict</code> refuses to install on anything else rather than warning and continuing                                                                                     |

Not used, deliberately: no state-management library (one state machine, held in one hook), no CSS framework (the design deliverable is a small custom surface), no i18n library (§10), no `ffmpeg` (§6), no native keyboard hook (§12).

## 12. Testing tiers, and what is cut

| tier                                             | what it covers                                                                           | cost                            |
|--------------------------------------------------|------------------------------------------------------------------------------------------|---------------------------------|
| domain unit tests                                | the turn machine, the 250 ms rule, WAV encoding, reply parsing                           | no Electron, milliseconds       |
| adapter tests                                    | argv construction, exit-code mapping, <code>ENOENT</code> → setup failure, timeout kills | spawns stubs, not real binaries |
| architecture test                                | the dependency rule, proven by breaking it                                               | ~15 lines                       |
| one Playwright <code>_electron</code> smoke test | the app launches, the preload bridge exists, one round trip crosses the real IPC seam    | one launch, seconds             |

**The cut, with its price named:** there is no broad end-to-end suite, no packaged-artifact test and no visual regression tier.

After this cut, the classes of defect running against **no** check at all are: a failure that only appears in a **packaged** (asar, signed, sandboxed) build, and any regression in the UI's appearance. What is *not* uncovered is "nothing executes in the renderer" — the single Playwright launch exists precisely so that a CSP or preload-wiring defect, which no unit test can see, still has one gate in front of it.

The native global keyboard hook is cut the same way: hold-to-talk works while the window has focus, so **a hold started while another app is focused does nothing**, and nothing detects that regression except using the app. The alternative costs a native module rebuilt per Electron version plus an Accessibility permission the user grants in System Settings — real money against a requirement the brief states as "hold a key **or** button".

## 13. Packaging

**Decided: out of scope for this delivery.** The whole promise is `npm run dev` on a clean machine, which is exactly what the README documents and exactly what gets tested. Packaging, signing, notarisation and a CI/CD pipeline are listed as future work at the end of `docs/WORK-BREAKDOWN.md`.

After this cut, the class of defect running against no check is anything that only appears in a **packaged** build — an `asar` path assumption, a spawned binary that is not in the bundle, a missing `Info.plist` key. Nothing in this repository exercises a packaged artifact, so that whole class arrives, if it arrives, on the day someone first runs `electron-builder`.

The microphone usage description (`NSMicrophoneUsageDescription`) is written into the build configuration anyway. It costs one line, and without it a future packaged build fails `getUserMedia` with no prompt and no explanation — which is the least debuggable failure in this entire application.

## 14. Decisions taken

---

The three questions this plan opened, and their answers:

1. **Default agent CLI** — `claude -p`, tested against `2.1.263`. `codex exec` and `cursor-agent -p` remain a second adapter each, not a change to the use case.
2. **Spoken reply — in scope**, via the macOS `say` binary behind `SpeechSynthesizer` (§6).
3. **Default Whisper model** — `base.en` (~150 MB). It keeps first-run setup to seconds, which is what the "runs on a clean machine" criterion is actually measuring. `large-v3-turbo` is one `VOICEDesk_WHISPER_MODEL` away and the README says so.
4. **Packaging — out of scope** (§13); dev build only, future work recorded.

# VoiceDesk — work breakdown

The build, decomposed before any of it is written. Each subtask carries a scope, an acceptance criterion **quoted from the client brief** (not "tests are green"), a complexity rating 1–10 that decides how much review it gets, and a parallelism label.

Status values: `PLANNED` · `IN PROGRESS` · `BLOCKED` · `DONE` (accepted) . Nothing is reported done until its acceptance criterion has been checked against the brief — and the umbrella is done only when every subtask is, not when most of them are.

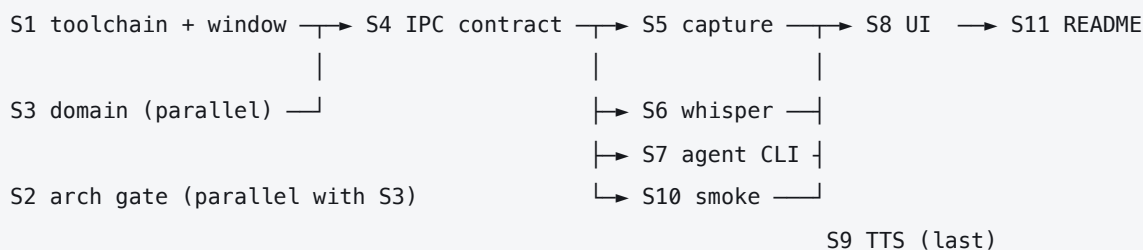
## Branching, and a deliberate deviation

The usual rule here is one branch per subtask, squash-merged. **This build commits directly to `main` instead**, because the brief grades the commit history itself:

*"Don't clean up the history. Real commits as you go, not one squash at the end."*

A squash merge per subtask is exactly the cleanup that sentence forbids. The trade is that `main` carries intermediate states; that is the point.

## Dependency shape



## Subtasks

### S1 — toolchain and a window that opens

Scope: stand up Node 26 / TypeScript strict / electron-vite with three build outputs, and a single `BrowserWindow` that opens with the three security switches written explicitly and a version badge reading `package.json` .

- **Acceptance:** *"It runs on a clean machine following your README"* — `npm ci && npm run dev` on a machine that has never seen this repo opens a window showing `v0.1.0 · dev` .

- **Complexity:** 5 · **Parallelism:** SEQUENTIAL (root) · **Status:** PLANNED

## S2 — the architecture gate

Scope: Vitest, plus the ~15-line test that fails when `domain/` imports the platform, proven by breaking it once.

- **Acceptance:** the gate has been *seen* to redden on a deliberate violation, and asserts it scanned a non-empty file list.
- **Complexity:** 3 · **Parallelism:** PARALLEL (with S3) · **Status:** PLANNED

## S3 — the domain

Scope: `TurnState`, `TurnFailure`, the three ports, and `runVoiceTurn` as a function taking ports as arguments. Unit tests for the turn machine, including the 250 ms rule and the "key repeat is ignored" rule.

- **Acceptance:** the whole flow *audio* → *transcript* → *reply* is expressible and testable with no Electron, no binaries and no network.
- **Complexity:** 5 · **Parallelism:** PARALLEL (needs nothing from S1) · **Status:** PLANNED

## S4 — the IPC contract

Scope: `shared/ipc.ts` (channel names + zod schemas, one declaration), the preload bridge with one named method per message, and main handlers that unwrap → call the use case → wrap. Errors cross as typed data.

- **Acceptance:** the renderer can reach exactly the five declared messages and nothing else; `ipcRenderer` is not exposed and no bridge function takes a caller-supplied channel name.
- **Complexity:** 6 · **Parallelism:** SEQUENTIAL (after S1, S3) · **Status:** PLANNED

## S5 — push-to-talk capture

Scope: hold-to-talk on the button and on `Space`; `getUserMedia` on first hold; `AudioContext` at 16 kHz; an `AudioWorklet` producing mono Float32; a live level meter; all four hold-ending events; microphone permission handled as three distinct outcomes.

- **Acceptance:** "Hold a key or button, speak, release → your words appear as text in the window."
- **Complexity:** 7 · **Parallelism:** SEQUENTIAL (after S4) · **Status:** PLANNED

## S6 — the transcription adapter

Scope: `WhisperCppTranscriber` over `execFile`, the pure Float32 → 16-bit PCM WAV encoder, binary + model resolution that survives a Finder-launched app's minimal `PATH`, and the `ENOENT` → *setup failure* mapping.

- **Acceptance:** speech becomes text with no API key, no account and no network; a missing binary or model says which one and how to install it.

- **Complexity:** 6 · **Parallelism:** PARALLEL (with S5, S7) · **Status:** PLANNED

## S7 — the agent CLI adapter

Scope: `ClaudeCliAgentRunner` and nothing else — `claude -p` **only**. Argv array via `execFile` with no shell, the full flag set from `docs/PLAN.md §5.2`, zod-parsed JSON reply, session continuity carried explicitly, timeout that kills, and the three-way outcome mapping. No second CLI adapter is written; the port already makes one cheap when it is wanted.

- **Acceptance:** "add milk to my shopping list" creates or edits `notes/shopping.md`, and "what's on my list?" answers from it — through the CLI, not the raw API, with the agent's blast radius bounded to `notes/`.
- **Complexity:** 7 · **Parallelism:** PARALLEL (with S5, S6) · **Status:** PLANNED

## S8 — the interface

Scope: implement the approved design — the eight states, the talk control with its meter, the four failure screens, the about/credits sheet, every string through `t()`.

- **Acceptance:** "UI: generate it with Claude Design ... it looks intentional and is usable", and each artboard state is recognisable in the running app.
- **Complexity:** 7 · **Parallelism:** SEQUENTIAL (after S5) · **Status:** BLOCKED — waiting on the design canvas being filled in and approved.

## S9 — spoken reply (*stretch*)

Scope: `MacSaySynthesizer` behind the port, plus the control that triggers it.

- **Acceptance:** "Stretch, only if you're inside the budget: the reply is spoken back."
- **Complexity:** 3 · **Parallelism:** PARALLEL (last) · **Status:** PLANNED — confirmed in scope. It is the cheapest point the brief offers and it stays cheap: one adapter, one control, no new dependency.

## S10 — the one end-to-end

Scope: a single Playwright `_electron` launch that starts the real app, asserts the preload bridge exists, and puts one round trip across the real IPC seam.

- **Acceptance:** something in this project executes in the renderer, so a CSP or preload-wiring defect has a gate in front of it.
- **Complexity:** 4 · **Parallelism:** SEQUENTIAL (after S4, S5) · **Status:** PLANNED

## S11 — the README

Scope: install from zero on a Mac that has nothing; run; what works; what was cut and what that stops catching; how long it actually took; and the full third-party licence list with names and versions, which is also the data behind the about panel.

- **Acceptance:** *"It runs on a clean machine following your README" and "A README that tells the truth".*
  - **Complexity:** 4 · **Parallelism:** SEQUENTIAL (written incrementally from S1 onward, closed last) · **Status:** IN PROGRESS
- 

## How each subtask is run

---

Complexity decides the shape of the loop, not the mood:

| rating | loop                                                                           |
|--------|--------------------------------------------------------------------------------|
| < 2    | done in the main thread, no ceremony                                           |
| 2–6    | develop → test → review, one reviewer                                          |
| ≥ 7    | develop → test → review, then a panel of independent critics before acceptance |

S5, S7 and S8 are the three that earn the panel. S4 and S6 sit just under it and get a full single review. Nothing is marked done on a green test alone — the gate is the acceptance criterion above it, checked against the brief.

---

## Answered before the build started

---

1. **Default Whisper model** — `base.en` (~150 MB). Fast first run beats accuracy on a criterion that is literally "it runs on a clean machine following your README". `large-v3-turbo` stays one environment variable away.
  2. **Spoken reply (S9)** — in scope, via the macOS `say` binary.
  3. **Packaging** — not in this delivery. `npm run dev` is the whole promise, and the cost of that cut is written into `docs/PLAN.md` §13 rather than left implied.
- 

## Future work — not in this delivery

---

Recorded so the absence is a decision rather than an oversight. None of it is started, none of it is promised, and the README does not claim any of it.



| #  | item                                                                                                                                                                                | why it is not now                                                                                                                |
|----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| F1 | <b>Packaged build</b> — <code>electron-builder</code> producing <code>.app</code> and <code>.dmg</code> , with <code>NSMicrophoneUsageDescription</code> in <code>Info.plist</code> | the delivery is a dev build; nothing in the repo exercises a packaged artifact today                                             |
| F2 | <b>Code signing, notarisation, stapling</b>                                                                                                                                         | needs a paid Apple Developer account and certificates on the build machine — an external dependency, not an engineering decision |
| F3 | <b>CI</b> — typecheck, unit tests, the architecture gate and the one Playwright launch on every push                                                                                | the gates exist and run locally; wiring them to a runner is a day's work that buys nothing until more than one person commits    |
| F4 | <b>CD</b> — tagged release publishing the signed artifact                                                                                                                           | strictly after F1–F3; a release pipeline without a signed artifact publishes something nobody can open                           |
| F5 | <b>Auto-update</b>                                                                                                                                                                  | only meaningful once F4 exists                                                                                                   |
| F6 | <b>Second agent adapter</b> ( <code>codex exec</code> , <code>cursor-agent -p</code> )                                                                                              | the port exists from day one, so this is an adapter and a line in the composition root — cheap later, wasted now                 |
| F7 | <b>Second language</b>                                                                                                                                                              | the lookup table and <code>t()</code> ship in this build; a locale is a JSON file and a switch                                   |
| F8 | <b>Global hold-to-talk</b> while another app is focused                                                                                                                             | needs a native keyboard hook rebuilt per Electron version plus an Accessibility permission — see <code>docs/PLAN.md</code> §12   |